

HelixConstitution

HelixConstitution

Әр жобаның мұраға алатын әмбебап инженерлік конституциясы — бұлфқа қарсы заң, механикалық түрде орындалатын, бір Git субмодулі ретінде бөлісілетін.

Қысқаша мазмұны

HelixConstitution — бұл бір ғана, жобаға тәуелсіз ережелер жинағы. Ол әрбір Helix/vasic-digital жобасына Git субмодулі ретінде қосылып, келіспейтін инженерлік тәртіпті (бұлфқа қарсы, дәлелдермен тексеру, деректер мен хост қауіпсіздігі, құжаттама және тест қамтуы) кодтап, оны 140-тан астам репозиторийлер флотына таратады. Бұл — бүкіл жүйенің тұтастығын қамтамасыз ететін басқару негізі.

Қысқаша сипаттамасы

Git субмодулі ретінде жеткізілетін әмбебап, мұраға алынатын Constitution. Ол әрбір тұтынушы жобаға автоматты түрде мұраға алатын және кеңейтуге, бірақ әлсіретуге болмайтын міндетті, келіспейтін ережелерді анықтайды: бұлфқа қарсы дәлелдік шлюздер, жалған оң нәтижелерге төзімділік, деректер мен хост қауіпсіздігі, қамту және құжаттама тәртібі.

Толық сипаттамасы

HelixConstitution — бұл Git субмодулі ретінде қосылуды таңдаған әрбір жобаның инженерлік тәжірибесіне арналған жалғыз, канондық шындық көзі — инженерлік заң, код сияқты таратылып, нұсқаға бекітілген. Оның негізгі бөлігі — Constitution.md — шамамен 1 МБ көлеміндегі, нөмірленген тармақтардан тұратын (қазіргі уақытта §11.4.170 дейінгі §11.4.x шарттар отбасы) және әрбір агенттің жұмыс нұсқаулары (CLAUDE.md, AGENTS.md, QWEN.md, GEMINI.md) осы құжатқа сілтеме жасайтын үздіксіз нұсқаланатын құжат. Осылайша

адамдар мен әрбір CLI агенті бірдей ережелер жинағынан оқиды. Мұра алу үш деңгейлі болып жобаланған: әмбебап негіз (осы субмодуль), жоба деңгейі (жобаның өз Constitution/CLAUDE/AGENTS құжаттары, оны кеңейтетін) және міндетті емес әрбір субдиректория деңгейі — жоғарыдан төмен қарай бағаланады, мұнда жоба ережелерді *қатаңдата* алады, бірақ оларды *әлсіретуге* сәулеттік түрде тыйым салынады. Нәтижесінде 140-тан астам репозиторийлер флоты үнсіздей ажырауы мүмкін емес, өйткені олардың ұстанатын тәртібі бекітілген, еске сақталмаған.

Құжат абсолютті түрде салаға тәуелсіз: нақты сатушыны, аппараттық SKU-ны, портты немесе кітапхана нұсқасын атау кез келгені жобаның өз Constitution құжатына көшірілуі керек, ал әмбебаптық ешқашан болжам ретінде қабылданбайды — ол негізге енгізілмес бұрын төрт бөліктен тұратын анық сынақтан өтуі керек. Оның философиялық негізі — бұлфқа қарсылық, ол өзара байланысты шарттар отбасы ретінде көрініс табады: §1.1 жалған оң нәтижелерге төзімділік, §11.4 соңғы пайдаланушы сапасы шарты, §11.4.6 болжауға тыйым, §11.4.69 оң дәлелдер таксономиясы. Олардың біріктірілген әсері — жеткізудің өлшемі ешқашан “тесттер өтті” болмайды, ол “нақты пайдаланушы мүмкіндікті пайдалана алады”, ал әрбір жасыл нәтиже физикалық дәлелдерді келтірсе ғана есептеледі. Қосымша submodules-catalogue.md (142 репозиторий) “бізде бұған арналған нәрсе бар ма?” сұрағын каталог алдымен, қайта жасама, кеңейту рефлексіне айналдырады, бұл жаңа кодтың бір жолын жазу алдында орындалады. Қосымша скрипттер субмодульді кез келген кіріктіру тереңдігінен тауып, әрбір коммитті төрт тәуелсіз Git провайдеріне таратады, сондықтан бір ғана бекітілген ережелер жинағы жоғалту мүмкін емес.

Мазмұны

Неге құрастырдық

Бір иесі жазған бірнеше ірі өнімдік қолданбалар мен ондаған бөлек қайта пайдаланылатын кіші модульдер бірдей қиындықпен табылған ережелерді қайта қайта қайталаумен болды — және бірдей жіберіп алдау түрін жиі кездестірді: тесттер мен статус есептері сәттілік туралы мәлімдесе де, соңғы пайдаланушы үшін мүмкіндік бұзылған (“PASS-алдау” және “FAIL-алдау”). Constitution-дағы әрбір соттық бекітпе нақты оқиғаға байланысты (мысалы, 2026-05-20 D3 дыбыс бағыттау PASS-алдауы, мұнда “Қолданылып жатқан кодек” өрісі бос болғанымен тексеру жасыл түске ауысқан, немесе 2026-06-25 алып түймелі UI, мұнда нақты экран бұзылғанымен токен теңдігі тесттері сәтті өткен). Constitution осы алдаудың

бүкіл түрін бір рет, әмбебап түрде механикалық жолмен мүмкін емес ету үшін бар — бұл тәртіп жобалары арасында ауытқымауы немесе тыныштықпен ұмытылмауы үшін.

Неліктен ол ойынды өзгертеді

Бұл инженерлік мәдениетті құжаттама-үмітпен-қолданылатын нәрседен мұраланған, нұсқаланған, механикалық түрде мәжбүр ететін заңға айналдырады — бұл стиль нұсқаулығы мен компилятордың айырмашылығы. Бір кіші модульді жаңарту ережелерді бүкіл флоты үшін бірден, атомдық және іздеуге болатын түрде жаңартады. Бір алдауға қарсы шарт барлық тұтынушы репозиторийлерде сенімге емес, құрылымға байланысты *кепілдендірілген*: тарату қақпасы флоты бойынша шарт нөмірін әдеби түрде іздейді, ал онымен бірге жүретін мутациялық тест қақпаның өзі алдамайтынын дәлелдейді — тіпті бақылаудың өзі де бақыланады. Басқару енді викиде ешкім оқымайтын ұмтылыс емес, CI жұмысына көрсете алатын аудитке жатқызылатын, тексерілетін факт болып шықты.

Қандай жаңалықтары бар

- **Constitution-кіші модуль ретінде** — инженерлік заң код сияқты таратылып, нұсқаға бекітіледі, әдейі v1.0.0 үлгісіндегі белгілермен және жоба бойынша бекітумен, сондықтан әрбір репозиторий қандай заң нұсқасына байланысты екенін *дәл* біледі.
- **Алдауға қарсы бірінші дәрежелі, соттық доктрина ретінде** — әрбір шарт тура мақсатты оператор талабына және жиі оны тудырған нақты оқиғаға сілтейді, сондықтан ережелер жинағы пікір емес, іс жүргізу заңы сияқты оқылады.
- **Ережелердің өзін мета-тесттеу (§1.1)** — әрбір қақпа онымен бірге жүретін мутациямен жұптастырылады, ол PASS→FAIL ауысуы керек, сондықтан “қақпа жалған емес” деп тек мәлімдеу емес, әрбір іске қосуда дәлелденеді; ешқашан сәтсіз бола алмайтын қақпа қақпасыз жағдайдан да нашар деп саналады.
- **Жеткен әмбебаптық** — ереже әмбебап па, әлде жобаға ғана тән па екенін анықтау үшін анық төрт бөліктен тұратын тест қолданылады, бұл базалық бөлікті жеңіл, тасымалданатын және сатушының әсерінен азат ұстайды.

Барлық өнімдерде қалай қолданылады (ол беретін мүмкіндіктер)

Міндетті басқару тірегі ретінде HelixConstitution отбасының кеңес алатын құжаты емес — ол отбасының құрылған негізі:

- **Басқару тірегі:** әрбір Helix/vasic-digital жобасы оны кіші модуль ретінде қосады және CLAUDE.md / AGENTS.md / QWEN.md немесе өзіндік Constitution.md файлынан импорттайды; ережелер шартсыз қолданылады, бірінші коммиттен бастап, жоба бойынша бас тарту мүмкін емес.
- **Қақпалар мен талаптар:** ол төрт деңгейлі қамту моделін анықтайды — көздер бар, құрастырудан өтеді, орындалу уақытында әрекет етеді, қақпа алдамайды — мүмкіндіктің барлық төрт деңгейден өтуі керек, содан кейін ғана ол аяқталды деп есептеледі, сонымен қатар атаулы талаптардың өсуі: аккредитацияларды өңдеу (§11.4.10), құжаттама әрқашан синхрондалған (§11.4.60), контейнерлер-кіші модуль талабы (§11.4.76), CodeGraph (§11.4.78), міндетті тест түрлерінің қамтылуы (§11.4.169) және тағы басқалары.
- **Тарату:** CM-COVENANT-114-NNN-PROPAGATION қақпалары тұтынушы флоты бойынша дәл шарт мәтінінің бар екенін растайды, сондықтан шарт иеліктің бір бұрышында тыныштықпен алып тасталуы мүмкін емес; үйлеспеушілік ешқандай құтылу флагтарымен өткізілмейтін қатаң шығару бөгеті болып табылады.
- **Ашу:** submodules-catalogue.md “бізде X-ті жасайтын нәрсе бар ма?” деген сұраққа бір қарап жауап беруге мүмкіндік береді, бұл кез келген жаңа модульді құрастыру алдында қосарланған күш жұмсауды бастапқыдан жояды.
- **AI агенттерінің біркелкілігі:** бірдей заң әрбір CLI агентіне бірдей түрде беріледі (Claude Code, Codex/Cursor/Aider/OpenCode/Crush/Kimi AGENTS.md арқылы, Qwen Code QWEN.md арқылы), сондықтан кодқа қандай құрал тигеніне қарамастан, ол бір ғана шартқа бағынады.

Мазмұн

Ең үлкен техникалық қиындықтар және оларды қалай шештік

- **Кез келген кірістірілген тереңдіктегі кіші модульді табу** — үш кіші модуль тереңдігінде жасырылған ереже әлі де заңды табуы керек, оның қай жерде екенін білмей-ақ → find_constitution.sh ата-аналық директориялар бойынша жоғары қарай жүріп, git супержоба көрсеткішін рекурсивті

түрде бақылайды, CONSTITUTION_DIR ауыстыруын және екі қолдаулы пішімді (constitution/, submodules/constitution/) қабылдайды, сондықтан кірістіру тереңдігі қандай болса да шешім анық болады.

- **Бір репозиторийдің төрт Git провайдеріндегі басымдығын сақтау** — айналар айырмашылыққа ұшыраса, пайдасыз → `install_upstreams.sh` декларативті `Upstreams/*.sh` қашықтағы репозиторийлерді оқып, `origin`-ды бірнеше `push URL`-мен конфигурациялайды, сондықтан бір `git push` GitHub (негізгі), GitLab, GitFlic және GitVerse-ке атомдық түрде таралады, ешқандай айна артта қалмайды.
- **Жалпы базаға ережелердің көбеюі мен жобалық ақпараттың өтуін болдырмау** — әрбір “мұны осында қосып қойсақ” деген азғындату портативтілікті бұзады → әрбір жаңа ережеге жалпылыққа ие болудың төрт бөліктен тұратын сынағы мен §11.4.17 жалпылық-жобалық жіктелуі қолданылады, жобаға тән мәселелерді оларға тиесілі жобалық деңгейге кері қайтарады.
- **Мұрагерлік қақпаның шынымен жұмыс істейтінін дәлелдеу** — сіз көрмеген қақпа сенімді емес → `meta_test_inheritance.sh` қадағалаушы мета-тест §11.4 тірегін әдейі жойып, қақпаның оны ұстайтынын растайды, сондықтан бақылау механизмі өзі үнсіз бұзылуға қарсы үздіксіз қайта тексеріледі.

Технологиялық стек

- **Git кіші модульдерінің мұрагерлігі** — *неліктен*: Git кіші модульдері — ережелер жинағының әрбір тұтынушы үшін беделді әрі нұсқаға бекітілген болуына, көшіріп-қояудың орнына анық, қарастырылатын жаңарту арқылы жаңартылуына мүмкіндік беретін жалғыз механизм; *қалай*: тұтынушы жобалар кіші модульді қосып, оның агенттік файлдарын `@import` арқылы импорттайды, ал үш деңгей жоғарыдан төмен қарай қатаң “кеңейтеді, әлсіретпейді” шартымен бағаланады.
- **`find_constitution.sh`** — *неліктен*: ережелер терең кірістірілген код оларды сенімді таба алмаса, пайдасыз, ал жолдарды қатаң кодтау жоба қайта ұйымдастырылған сәтте бұзылады; *қалай*: ата-аналық директориялар бойынша жүру плюс `git rev-parse --show-superproject-working-tree` рекурсиясы, CONSTITUTION_DIR ауыстыруымен қамтамасыз етілген, екі қолдаулы пішімді шешеді.
- **`install_upstreams.sh` + `Upstreams/`** — *неліктен*: төрт провайдерлік резерв тек оны сақтау үшін қосымша күш жұмсау қажет болмаса ғана нақты, әйтпесе айналар ескіреді; *қалай*: әрбір қашықтағы репозиторий үшін декларативті `.sh` файлдары бір URL `origin`-ға айналдырылып, төрт `push` бірге біріктіріледі.

- **§1.1 өзгеріс мета-тесттері** — *неліктен*: ешқашан істемейтін қақпа жоқ қақпадан да нашар, өйткені ол жалған сенім тудырады; *қалай*: әрбір қақпаға sed арқылы жою/атауын өзгерту мутациясы қосылып, ол PASS→FAIL ауысуы керек, содан кейін қалпына келтіріледі, сондықтан әрбір қақпа әр іске қосылғанда өз әсерін дәлелдейді.
- **Тарату қақпалары (CM-COVENANT-114-MNN-PROPAGATION)** — *неліктен*: шарт тек барлық тұтынушыларда тексерілетін болса ғана жалпылық, тек жетекші репозиторийде ғана емес; *қалай*: тұтынушылар бойынша тарау нөмірінің тікелей грер-і, оны §1.1 мутациясымен қамтамасыз етіп, тарату тексерісінің өзі істемей қалуы мүмкін екенін дәлелдейді.
- **submodules-catalogue.md (§11.4.74)** — *неліктен*: анти-көшіру тәртібін бұзудың ең жылдам жолы — сізде не бар екенін білмеу; *қалай*: 142 репозиторийден тұратын, мүмкіндіктер бойынша топтастырылған тізім, жаңа нәрсені құру алдында каталогтық тексеріс трекерге тіркеледі.
- **Көп пішімді экспорт** — *неліктен*: бірдей заң адамдардың оқуы, құралдардың талдауы және мұрағаттардың сақтауы үшін бірдей қолжетімді болуы керек; *қалай*: әрбір канондық құжат бір дереккөзінен .md / .html / .pdf / .docx пішімдерінде шығарылады.

Мазмұн

Жағдай және адалдық туралы ескертпелер

- **Жағдайы: жөнелтілген.** Белсенді түрде нұсқалануда және флотилиядағы (қоғамдық канондық және айналық репозиторийлер) кіші модуль ретінде қолданылып келеді.
- **Лицензиясы: анықталмаған** — қарастырылған бастапқы материалда айқын көрсетілмеген; жарияламас бұрын репозиторийдің LICENSE файлымен растаңыз.
- Қосымша жоғары ағымдағы айналар: GitLab helixdevelopment1/helixconstitution, GitFlic helixdevelopment/helixconstitution, GitVerse helixdevelopment/HelixConstitution.

Басымдық деңгейі: Helix-негізгі — Helix отбасындағы барлық нәрсенің құрылысына міндетті басқару тірегі.